

Двухбашенные нейросети II

ШАД, Рекомендательные системы
2025/26, лекция 5

Кирилл Хрыльченко

Старший преподаватель,
ФКН ВШЭ

План занятия

На лекции обсудим кодирование объектов в эмбединги:

1. Обучаемые эмбединги
2. Контентное кодирование
3. Unsupervised representation learning
4. Как кодировать пользователей

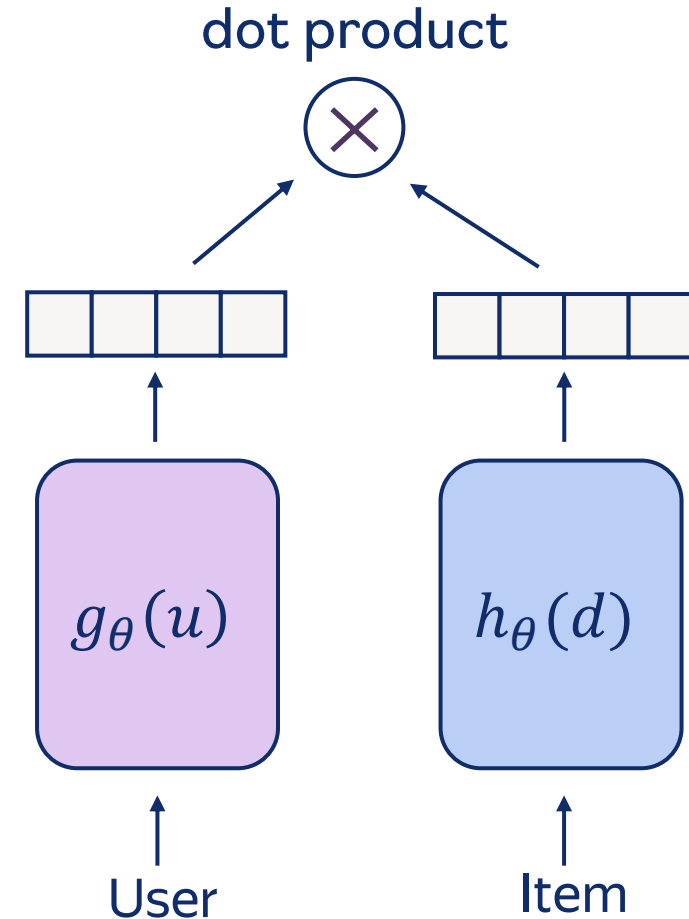
И уход от двухбашенных моделей в сторону более продвинутых функций близости.

На семинаре:

- Различные аспекты обучения нейросетевых моделей для рекомендательных систем

Recap

- На прошлом занятии обсудили базовую архитектуру и обучение двухбашенных моделей для нейросетевой генерации кандидатов
- На этом занятии обсудим архитектуру башен и продвинутые функции близости



Обучаемые эмбединги

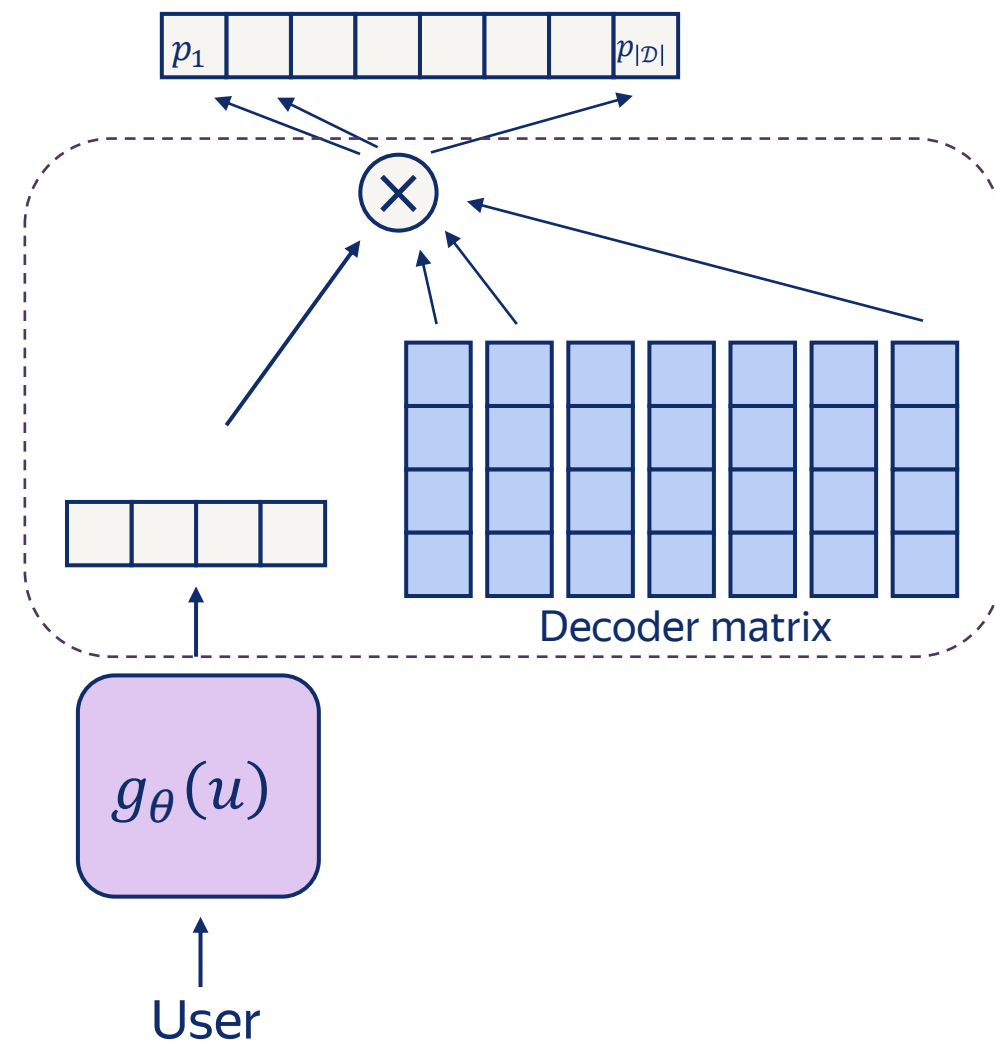
Часть 1

Обучаемые эмбединги

В softmax модели у нас естественным образом возникли **обучаемые векторы** айтеров, которые:

- Являются параметрами модели
- Оптимизируются вместе с остальными параметрами градиентным спуском
- ID-based embeddings

Где ещё в курсе мы сталкивались с обучаемыми эмбедингами?

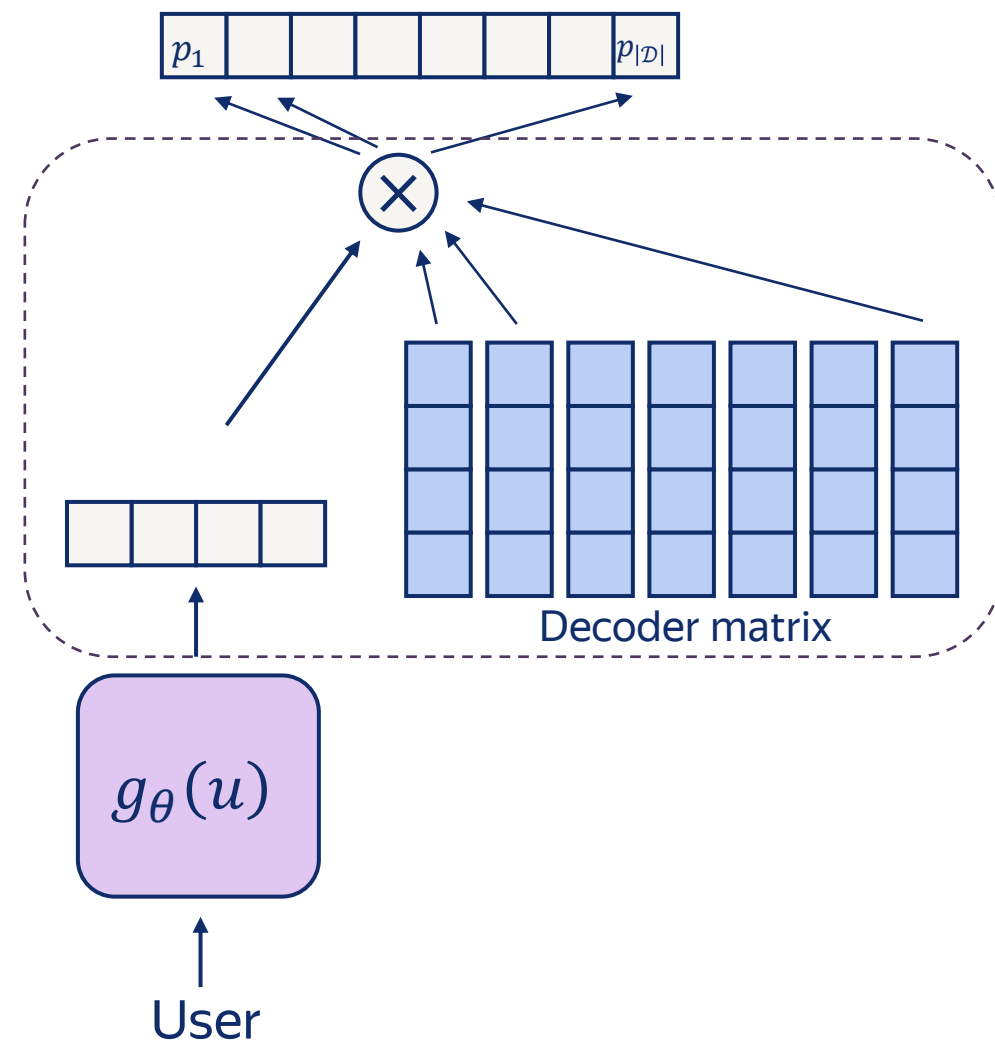


Обучаемые эмбединги

В softmax модели у нас естественным образом возникли **обучаемые векторы** айтеров, которые:

- Являются параметрами модели
- Оптимизируются вместе с остальными параметрами градиентным спуском
- ID-based embeddings

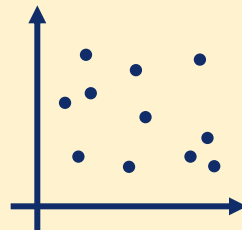
Где ещё в курсе мы сталкивались с обучаемыми эмбедингами? Матричная факторизация



Мощность обучаемых эмбеддингов

Обучаемые эмбеддинги – очень мощная модель:

- Можно выучить любую структуру пространства
- Если есть нейросетевой энкодер, то можем инициализировать обучаемые эмбеддинги из него
- А можно ли сделать наоборот?



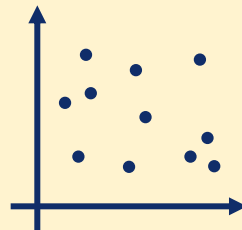
Эмбеддинги пользователей и айтеров лежат в едином семантическом пространстве.

Мощность обучаемых эмбеддингов

Обучаемые эмбеддинги – очень мощная модель:

- Можно выучить любую структуру пространства
- Если есть нейросетевой энкодер, то можем инициализировать обучаемые эмбеддинги из него
- А можно ли сделать наоборот? Дистилляция.

Но это только при условии отсутствия оси времени.



Эмбеддинги пользователей и айтемов лежат в едином семантическом пространстве.

Минусы обучаемых эмбеддингов

1. Проблема холодного старта
2. Плохое качество на тяжелом хвосте
3. Distribution drift
4. Переобучение
5. Нужно много памяти для хранения

Холодный старт

- Модель с обучаемыми эмбедами трандуктивна – область применения модели ограничена объектами из обучающей выборки
 - Если у нас появляются новые объекты, то для них нет эмбедингов
- Пусть у нас в каталоге появляются новые айтемы. Как их рекомендовать?

Холодный старт

- Модель с обучаемыми эмбедами трандуктивна – область применения модели ограничена объектами из обучающей выборки
 - Если у нас появляются новые объекты, то для них нет эмбедингов
- Пусть у нас в каталоге появляются новые айтемы. Как их рекомендовать?
 - Добавить отдельный источник кандидатов со свежим контентом (и надеяться, что наши следующие стадии – **индуктивны**)
 - Дообучить модель, как только накопим фидбек для новых айтемов

Тяжелый хвост и переобучение

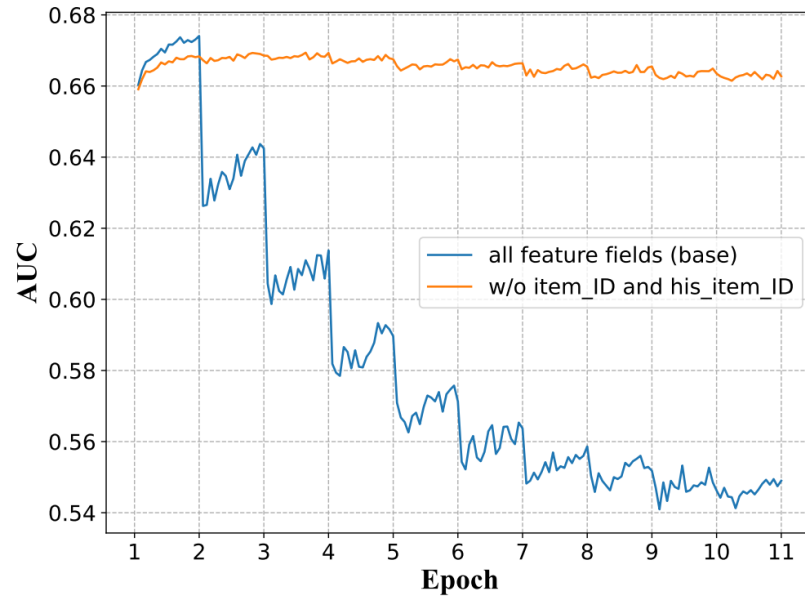


Figure 12: If sparse feature fields (item ID and history item IDs) are not used for training, the model will not experience the one-epoch phenomenon.

[Towards Understanding the Overfitting Phenomenon of Deep Click-Through Rate Prediction Models](#)

Тяжелый хвост и переобучение

ABSTRACT

ID-based embeddings are widely used in web-scale online recommendation systems. However, their susceptibility to overfitting, particularly due to the long-tail nature of data distributions, often limits training to a single epoch, a phenomenon known as the "one-epoch problem." This challenge has driven research efforts to optimize performance within the first epoch by enhancing convergence speed or feature sparsity. In this study, we introduce a novel

[Taming the One-Epoch Phenomenon in Online Recommendation System by Two-stage Contrastive ID Pre-training](#)

Тяжелый хвост и переобучение

Рекомендательные модели с обучаемыми эмбедингами начинают переобучаться после первой эпохи обучения.

Что такое переобучение?

- Подсказка: bias-variance tradeoff

Тяжелый хвост и переобучение

Рекомендательные модели с обучаемыми эмбедингами начинают переобучаться после первой эпохи обучения.

Что такое переобучение?

- Подсказка: bias-variance tradeoff
- Сложные модели запоминают шум в данных

Меморизация

Меморизация – это запоминание сэмплов вместо вывода общих закономерностей для нескольких сэмплов сразу.

Рассмотрим конкретный сэмпл из датасета:

- Обучим две модели – без сэмпла и с ним
- Замерим ошибку на сэмпле для обеих моделей
- Пусть разница ошибок – небольшая. Что это значит?

Меморизация

Меморизация – это запоминание сэмплов вместо вывода общих закономерностей для нескольких сэмплов сразу.

Рассмотрим конкретный сэмпл из датасета:

- Обучим две модели – без сэмпла и с ним
- Замерим ошибку на сэмпле для обеих моделей
- Пусть разница ошибок – небольшая. Что это значит?
 - Смогли обобщиться на этот сэмпл
- А если разница ошибок большая?
 - В датасете нет похожих сэмплов и мы его **запоминаем**

[What Neural Networks Memorize and Why: Discovering the Long Tail via Influence Estimation](#)

Меморизация

Теперь рассмотрим айтем, встретившийся в датасете один раз:

- Если исключить его из датасета – ошибка будет гигантская, потому что ничего про него не знаем
- Если добавить его в обучение, то данных недостаточно, чтобы понять каким пользователям он понравится
- Все, что мы можем – запомнить, что он встречался у конкретного пользователя

В тяжелом хвосте находятся айтемы, у которых мало сэмплов. Мы можем их запомнить, но сложно их порекомендовать.

Переобучение

- При обучении, модели сложно различать адекватные сэмплы из тяжелого хвоста и шум:
 - От меморизации тяжелого хвоста – вреда нет, кроме того, что тратим емкость модели на что-то не очень полезное
 - Когда запоминаем шум – наступает переобучение
- Если у модели сильные способности к меморизации и в данных много шума => будет переобучение
- Обучаемые эмбединги сильно увеличивают меморизацию

Distribution drift

- Реактивность: хотим быстро реагировать на изменения
 - E.g., пользователю категорически не понравился какой-то контент, или продавец улучшил описание товара
- Если учиться на пошафленных данных, то обучаемые эмбеддинги – это усредненные представления объектов за всё время
 - Нас интересует только актуальное состояние объекта на текущий момент времени
- Решение: учиться хронологически и дообучаться на новых данных
 - но возникают другая проблема – катастрофические забывание

Память

- Обучаемые эмбединги занимают **много памяти**:
 - 20 миллиардов обучаемых эмбедингов размерности 256 в float32 – это 19 терабайт
 - А если размерность эмбедингов выкрутить до 4096, как модно в LLM, то это 305 терабайт
- Сложно уместить обучаемые эмбединги и их статистики на GPU
 - Нужен сложный параллелизм, либо хранить эмбединги в RAM
 - А ещё есть hashing trick

Summary

Обсудили обучаемые эмбединги:

- Что это довольно мощная модель
- Но из-за специфики реальных данных (холодный старт, тяжелый хвост, шум, distribution drift) этой мощностью сложно воспользоваться

Довольно типичная ситуация для машинного обучения. Что делать?

Summary

Обсудили обучаемые эмбединги:

- Что это довольно мощная модель
- Но из-за специфики реальных данных (холодный старт, тяжелый хвост, шум, distribution drift) этой мощностью сложно воспользоваться

Довольно типичная ситуация для машинного обучения. Что делать? Вводить **inductive bias**.

Контентное кодирование

Часть 2

Inductive bias

Inductive bias – это ограничения на модель, увеличивающие обобщающую способность:

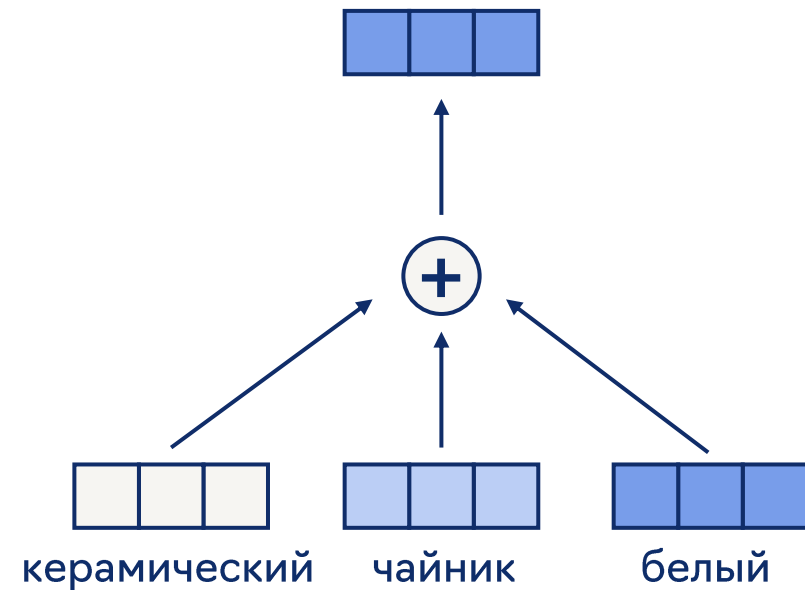
- Уменьшаем мощность модели
- Растим bias, уменьшаем variance (bias-variance tradeoff)
- Уменьшаем меморизацию
 - А значит, уменьшаем переобучение на шум
 - Вопрос: а что происходит с тяжелым хвостом?

Пример: мешок слов

Пусть в качестве айтемов товары, и для них есть названия:

- Токенизируем название
- Извлекаем для токенов обучаемые эмбединги
- Усредняем, чтобы получить единый вектор товара

Такую модель называют **мешком слов**.



Пример: мешок слов

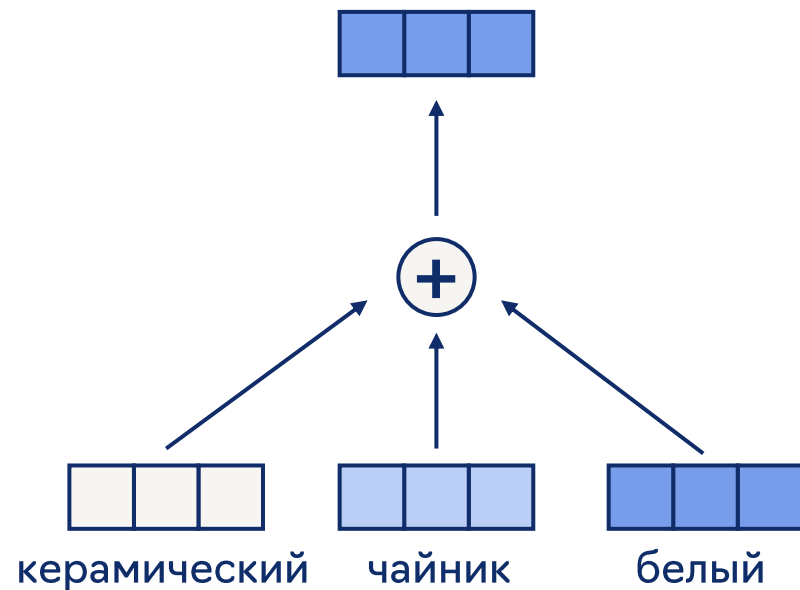
Предположения мешка слов:

- У товаров с похожими названиями должны быть похожие эмбединги
- Порядок слов не влияет на эмбединг

Мы ввели ограничения на структуру выучиваемого семантического пространства

- товары с похожими названиями должны быть рядом

Это inductive bias!



Примеры inductive bias

Допустим, используем обучаемые эмбединги.

Как увеличить inductive bias, не выходя за рамки обучаемых эмбедингов?

Примеры inductive bias

Допустим, используем обучаемые эмбединги.

Как увеличить inductive bias, не выходя за рамки обучаемых эмбедингов?

- Уменьшение размерности эмбедингов
 - Переходим в подпространство исходного семантического пространства
- l_2 -нормализация эмбедингов
 - Переходим в единичную гиперсферу
- Регуляризация эмбедингов при обучении

Примеры inductive bias

Пусть есть три варианта модели:

- Обучаемые эмбединги
- Энкодер над признаком A
- Энкодер над двумя признаками A и B

Где больше inductive bias?

Примеры inductive bias

Пусть есть три варианта модели:

- Обучаемые эмбединги
- Энкодер над признаком A
- Энкодер над двумя признаками A и B

Где больше inductive bias? $1 < 3 < 2$

- У обучаемых эмбедингов меньше всего inductive bias
- У модели с двумя признаками больше степеней свободы, чем у модели с одним

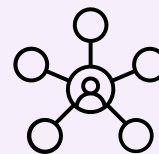
Контентное кодирование

Контентное кодирование – кодируем содержимое объектов:

- Название, описание, изображение, аудио
- Метаданные: артист, жанр; бренд, цена

С помощью энкодера:

- Average pooling эмбеддингов
- Трансформер над текстом, изображением; мультимодальный энкодер
- DCN, MLP над признаками
- Комбинация всего перечисленного



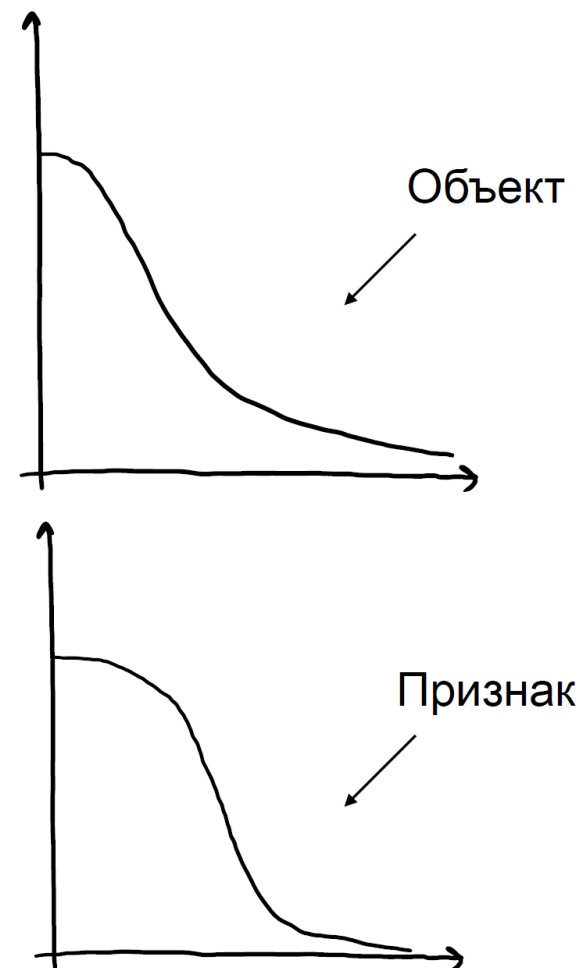
Если контентный энкодер обучается с использованием пользовательского фидбека, то эта модель и контентная, и коллаборативная.

Плюсы контентного кодирования

- **Индуктивность:** для новых объектов сразу получаем контентные эмбединги
- Если контент поменялся – можем сразу обновить эмбединг
- Уменьшаем меморизацию и переобучение
- Улучшаем обобщающую способность и качество на тяжелом хвосте
- Не нужно хранить гигантскую таблицу эмбедингов

Контентное кодирование тяжелого хвоста

- Признаки повторяются между объектами
- Если каждому объекту соответствует уникальное значение признака, то признак == item ID
- Тяжелый хвост признаков:
 - Он всегда короче, чем хвост айтемов – так как меньше уникальных значений
 - Он толще – так как больше сэмплов на каждое значение
- Контент увеличивает обобщающую способность



Минусы контентного кодирования

- Выразительность модели ниже, чем у обучаемых эмбеддингов:
 - Для популярных (и статичных) объектов обучаемые эмбеддинги очень хорошо работают
- Разменяли память на вычисления:
 - обучать и применять сложные энкодеры – вычислительно дорого

Case study: item representations

Статья от Google Research:

- Улучшают качество на тяжелом хвосте
- Поделили признаки айтемов на две группы:
 - Меморизующие (айдишники) и генерализующие (контент)
 - Кодируют их отдельными “экспертами”
- Frequency-based gating

$$y = \sum_{k=1}^{n_1} G(\mathbf{i})_k E_k^{mm}(\mathbf{i}_{mm}) + \sum_{k=n_1+1}^{n_1+n_2} G(\mathbf{i})_k E_k^{gen}(\mathbf{i}_{gen})$$

[Empowering Long-tail Item Recommendation through Cross Decoupling Network \(CDN\)](#)

Unsupervised representation learning

Часть 3

Supervised learning

- Хотим векторные представления пользователей и айтемов, которые хорошо решают нашу задачу
- Обычно мы смотрим на задачу как на **supervised learning**:
 - Задаём архитектуру модели и задачу обучения, учим всё “end-to-end”
 - Много сложностей и проблем, связанных со спецификой данных
- Это не единственный способ получить хорошие векторные представления объектов

Unsupervised representation learning

- Unsupervised learning – обучение на данных, которое не подразумевает наличия целевых меток
- Representation learning – обучение, в результате которого получаем представления объектов
- Self-supervised learning – превращаем unsupervised задачу в supervised с помощью искусственных целевых меток
 - Предобучение языковых моделей – self-supervised

Representation learning for RecSys

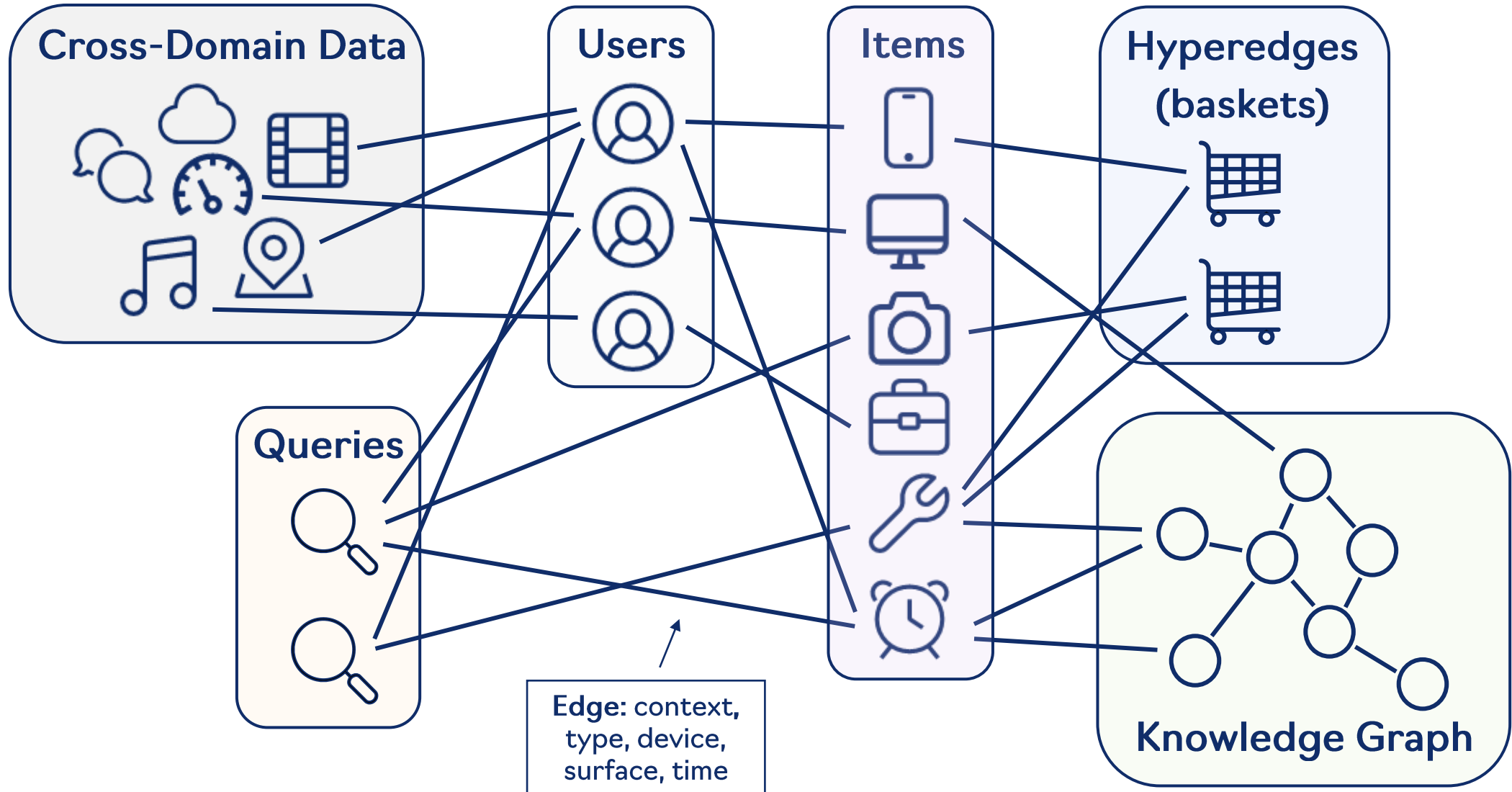
- Представляем рекомендательные данные в виде графа и получаем векторные представления вершин с помощью графовых моделей
- Обучаем векторные представления названий и изображений товаров с помощью **contrastive learning**
- Кодируем текстовые признаки товара с помощью SentenceT5 или EmbeddingGemma
 - Используйте модель, выдающую осмысленное векторное представление, а не рандомную LLM!

[Sentence-T5: Scalable Sentence Encoders from Pre-trained Text-to-Text Models](#)

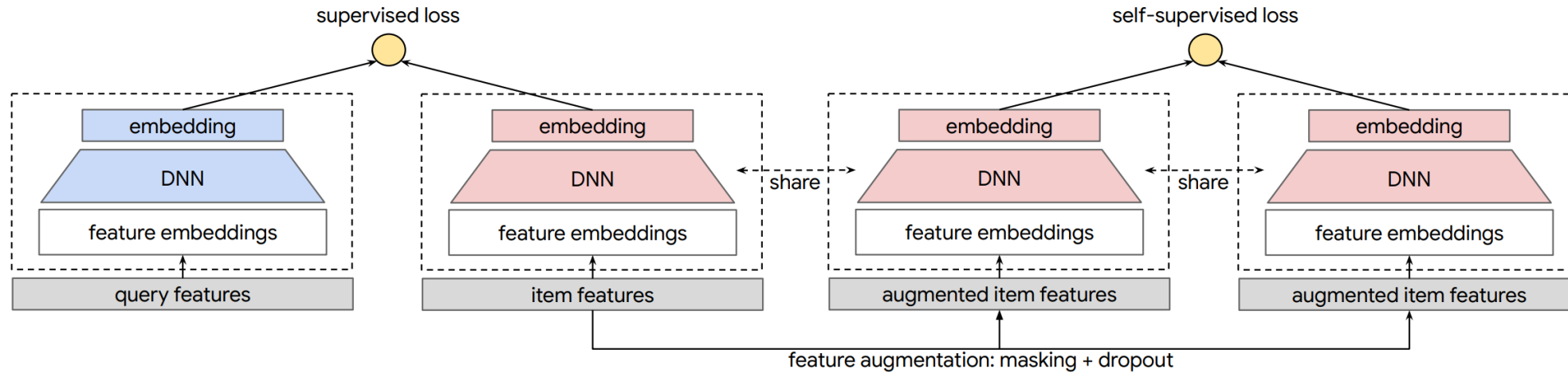
[EmbeddingGemma: Powerful and Lightweight Text Representations](#)

[Learning Transferable Visual Models From Natural Language Supervision](#)

Graph representation learning



Case study: self-supervised learning



Статья от Google Research:

- Опять улучшают качество на тяжелом хвосте при обучении двухбашенных моделей
- Добавили дополнительную self-supervised задачу:
 - Сближают векторы двух разных аугментаций признакового описания айтема и отдаляют чужие аугментации

[Self-supervised Learning for Large-scale Item Recommendations](#)

Кодирование пользователей

Часть 4

Кодирование пользователей

Обучаемые эмбединги плохо подходят для кодирования пользователей:

- Айтемы статичны, пользователи – нет
- Важна реактивность на фидбек пользователя, особенно на отрицательный
- Холодные пользователь очень важны
- Хотим context-awareness:
 - Учитывать в “запросной” башне не только пользователя, но и контекст – seed item, запрос, время, настройки рекомендаций, текущую корзину, etc

Обучаемый эмбединг пользователя

- Рассмотрим градиент по эмбедингу пользователя для двухбашенной модели и softmax loss:

Вектор пользователя – это линейная комбинация векторов айтемов, с которыми он взаимодействовал.

Item-to-item

Вспомним item-to-item:

- Формируем списки похожих айтемов на основе скалярных произведений item эмбедингов
- Агрегируем суммарные близости кандидатов к последним N положительным взаимодействиям пользователя $(d_1, \dots, d_n)$

Получается, что вектор пользователя – это усреднение векторов его последних N положительных взаимодействий.

Event pooling

- Формируем эмбединги последних N взаимодействий пользователя, затем делаем пулинг эмбедингов
- Если типов событий несколько, для каждого вводим обучаемый вес

Важное дизайн решение – как формировать векторы user-item взаимодействий:

- Обучаемые эмбединги айтемов (e.g., YoutubeDNN)
- Предобученные эмбединги (e.g., VideoBERT)
- End-to-end обучаемое кодирование (e.g., мешок слов)

Минусы event pooling

Не учитываем порядок событий:

- Недавние события должны иметь больший вклад в представление пользователя
- Не распознаем sequential patterns

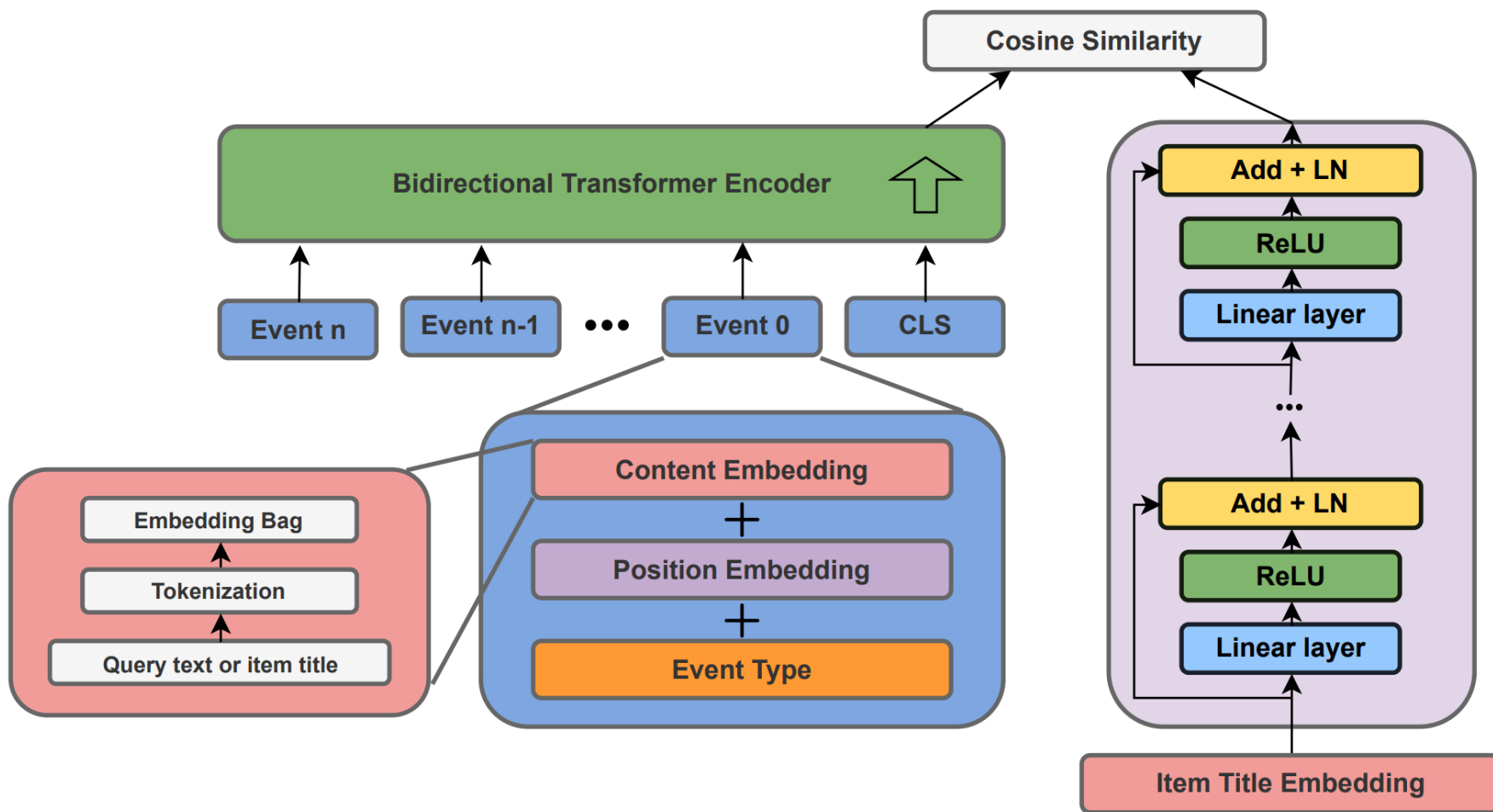
Time decay – взвешиваем события исходя из свежести.

Для второго пункта нам понадобятся более сложные sequential энкодеры.

Рекомендательные трансформеры

- Кодируем в вектор историю пользователя
 - Про каждое user-item взаимодействие знаем айтем, тип взаимодействия и его хронологическую позицию в истории
- Чтобы получить эмбединг события, кодируем в векторы айтем, тип взаимодействия и позицию, и складываем
- Применяем трансформер
- Делаем пулинг скрытых состояний из трансформера, чтобы получить вектор пользователя
 - CLS embedding, average pooling или последнее скрытое состояние

Рекомендательные трансформеры



Similarity Functions

Часть 5

Ограничения двухбашенных моделей

- Двухбашенная модель – это низкоранговое разложение матрицы релевантности
 - Ранг разложения, а значит и мощность двухбашенной модели, ограничивается размерностью эмбедингов
- Двухбашенные модели выдают неразнообразные рекомендации
 - Если в top K попал один айфон, то также попадут и все остальные айфоны в каталоге
- ANN-индекс – это сложно

Ограничения двухбашенных моделей

В Deermind поставили следующий эксперимент:

- Фиксируем количество документов N и K для top K
- Каждой возможной комбинации из K документов сопоставляется свой запрос
- Тюним обучаемые эмбединги запросов и документов размерности d на задачу предсказания правильных документов
- Для каждого значения d подбираем критический размер каталога, при котором не получается добиться идеальной точности

Ограничения двухбашенных моделей

Results Figure 2 shows that the curve fits a 3rd degree polynomial curve, with formula $y = -10.5322 + 4.0309d + 0.0520d^2 + 0.0037d^3$ ($r^2=0.999$). Extrapolating this curve outward gives the critical-n values (for embedding size): 500k (512), 1.7m (768), 4m (1024), 107m (3072), 250m (4096). We note that this is the best case: a real embedding model cannot directly optimize the query and document vectors to match the test qrel matrix (and is constrained by factors such as “modeling natural language”). However, these numbers already show that for web-scale search, even the largest embedding dimensions with ideal test-set optimization are not enough to model all combinations.

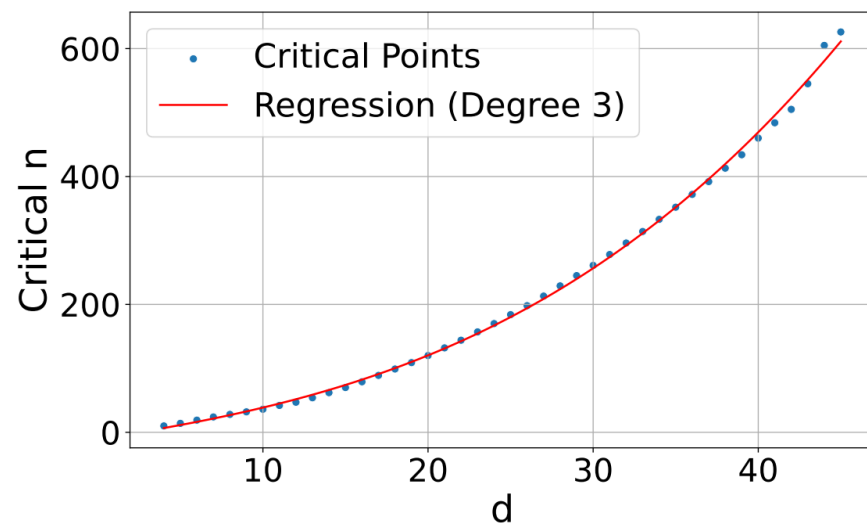


Figure 2 | The critical-n value where the dimensionality is too small to successfully represent all the top-2 combinations. We plot the trend line as a polynomial function.

Сложности ANN-индексов

Пусть мы (до)обучили двухбашенную модель:

- Нужно посчитать эмбединги всех айтемов
- Построить индекс на основе этих эмбедингов
- Подменить в продакшне индекс на новый
- А еще надо обновить модель в сервисе, считающем вектор запроса
 - Если есть рассинхрон индекса и модели в сервисе, качество может сильно упасть
- А еще нужно уметь обновлять индекс, когда появляются новые айтемы в базе

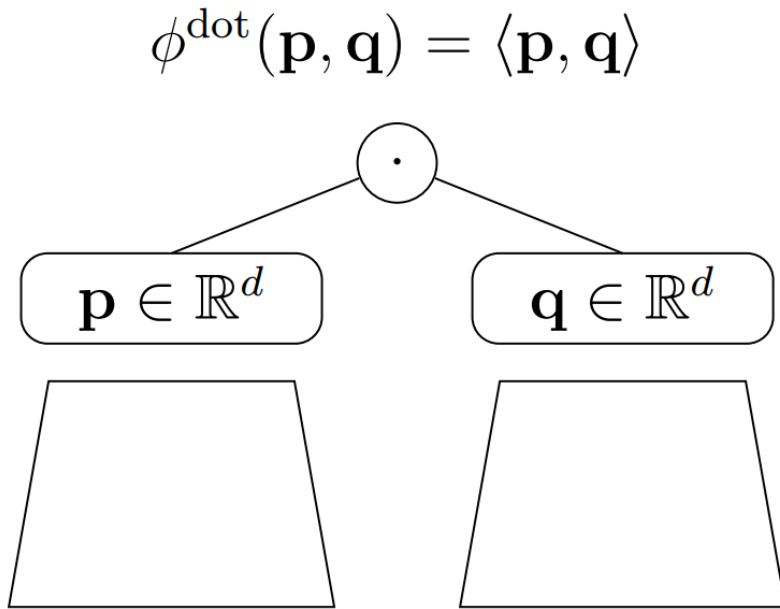
Ограничения двухбашенных моделей

Что хотим:

- Более мощную функцию близости
- Побороть проблемы с разнообразием
- Объединить расчет эмбеддингов и индекс в одном сервисе (одной модели)
– model-based подход

И при этом сохранить нужную для стадии генерации кандидатов эффективность и скорость.

MLP vs dot product



$$\phi^{\text{MLP}}(\mathbf{p}, \mathbf{q}) = \mathbf{f}_{W_l, \mathbf{b}_l}(\dots \mathbf{f}_{W_1, \mathbf{b}_1}([\mathbf{p}, \mathbf{q}]) \dots)$$

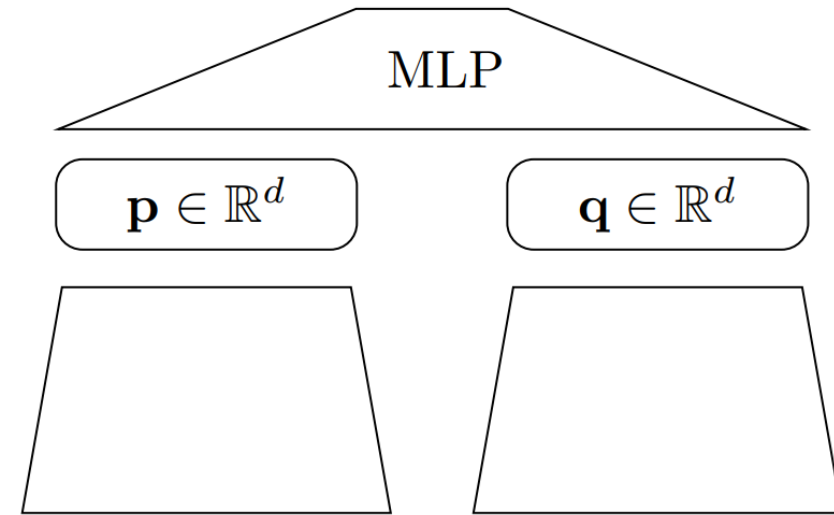


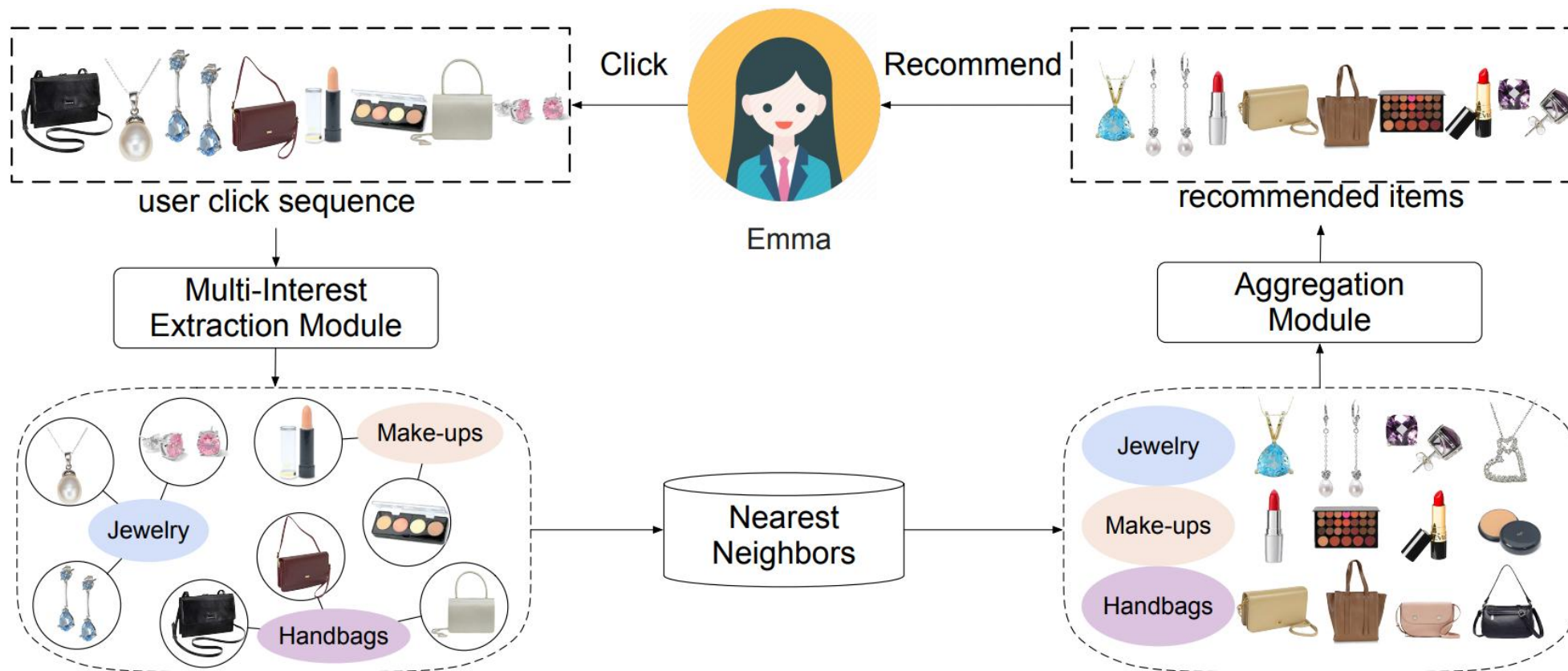
Figure 1: A model with dot product similarity (left) and MLP-based learned similarity (right).

MLP vs dot product

Abstract

Embedding based models have been the state of the art in collaborative filtering for over a decade. Traditionally, the dot product or higher order equivalents have been used to combine two or more embeddings, e.g., most notably in matrix factorization. In recent years, it was suggested to replace the dot product with a learned similarity e.g. using a multilayer perceptron (MLP). This approach is often referred to as *neural collaborative filtering* (NCF). In this work, we revisit the experiments of the NCF paper that popularized learned similarities using MLPs. First, we show that with a proper hyperparameter selection, a simple dot product substantially outperforms the proposed learned similarities. Second, while a MLP can in theory approximate any function, we show that it is non-trivial to learn a dot product with an MLP. Finally, we discuss practical issues that arise when applying MLP based similarities and show that MLPs are too costly to use for item recommendation in production environments while dot products allow to apply very efficient retrieval algorithms. We conclude that MLPs should be used with care as embedding combiner and that dot products might be a better default choice.

Multi-interest рекомендации



Multi-interest рекомендации

Пусть башня пользователя выдаёт несколько векторов. На обучении:

- считаем сразу M скалярных произведений
- агрегируем их в единый скор с помощью максимума или софтмакса

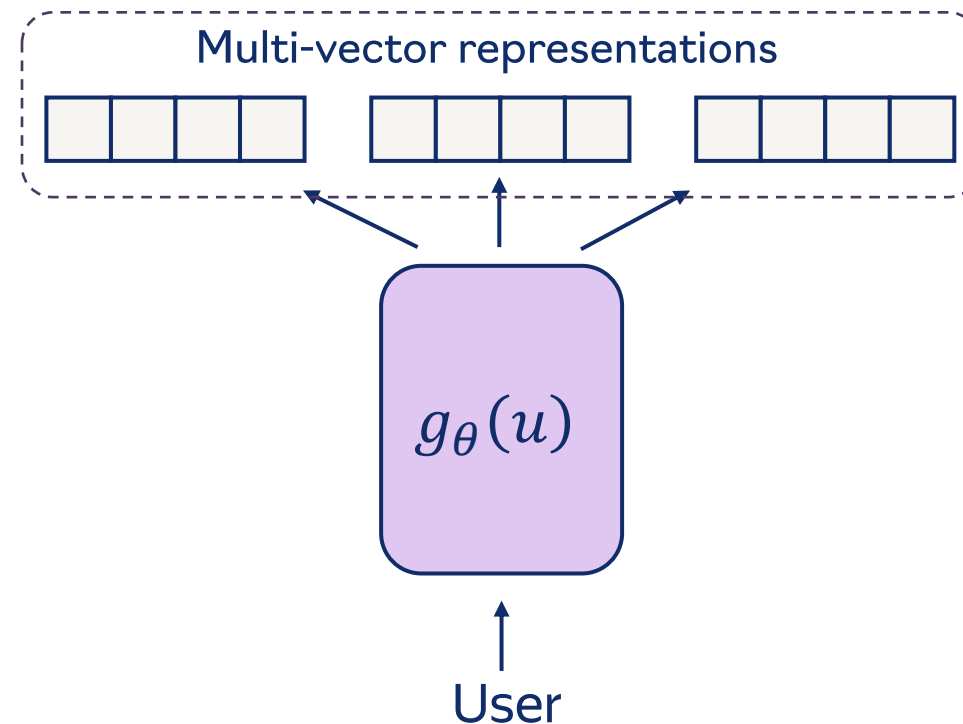
На инференсе:

- Считаем M эмбеддингов пользователя
- С каждым идём в индекс и вытаскиваем top K кандидатов
- Объединяем кандидатов и получаем финальный top K , посчитав агрегированный скор

Multi-interest рекомендации

Как получить несколько векторов пользователя:

- Капсульные нейросети
- Кластеризация взаимодействий
- Параметрический аттеншн
 - М обучаемых “глобальных” векторов
 - Аттеншн с глобальными векторами и векторами исторических событий
 - М CLS токенов на входе в трансформер
- Сделать один широкий эмбединг пользователя
 - И превратить в несколько эмбедингов

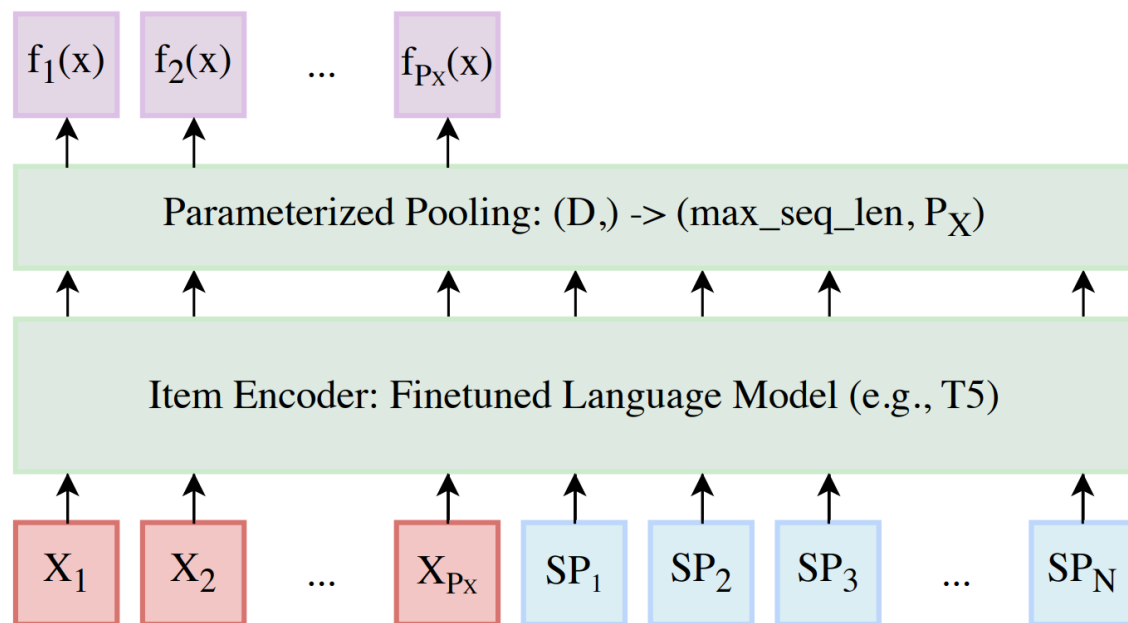


[Multi-Interest Network with Dynamic Routing for Recommendation at Tmall](#)

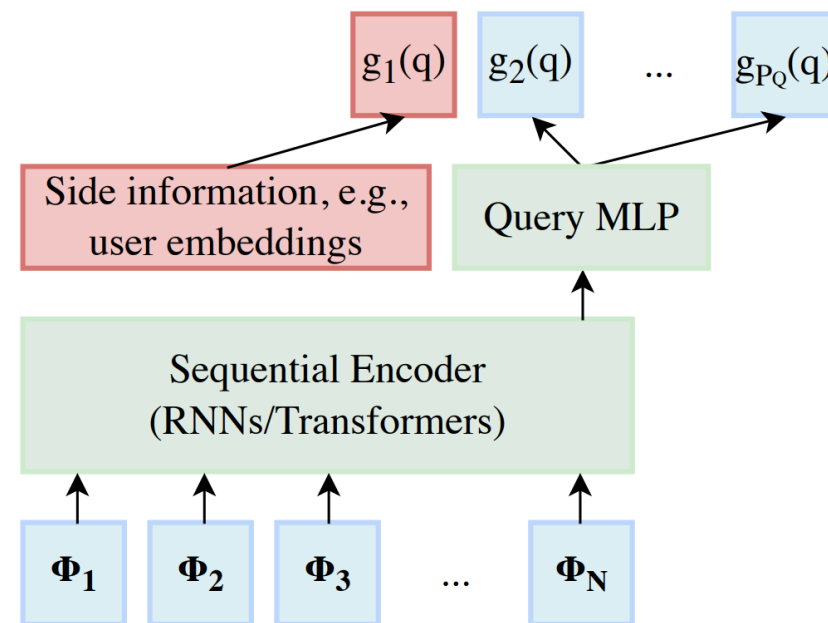
[Controllable Multi-Interest Framework for Recommendation](#)

[Density Weighting for Multi-Interest Personalized Recommendation](#)

Multi-interest рекомендации



Single, homogeneous feature (Language Models)



Rich, heterogeneous features (Recommendations)

Mixture of logits

- Можно сделать несколько векторов айтемов
- И добавить веса для логитов
- Смесь низкоранговых разложений – универсальный аппроксиматор
- На инференсе:
 - М разных индексов
 - В каждый идем со своим эмбедингом пользователя
 - Для $M * K$ кандидатов считаем итоговые скоры смеси и оставляем top K

GPU retrieval

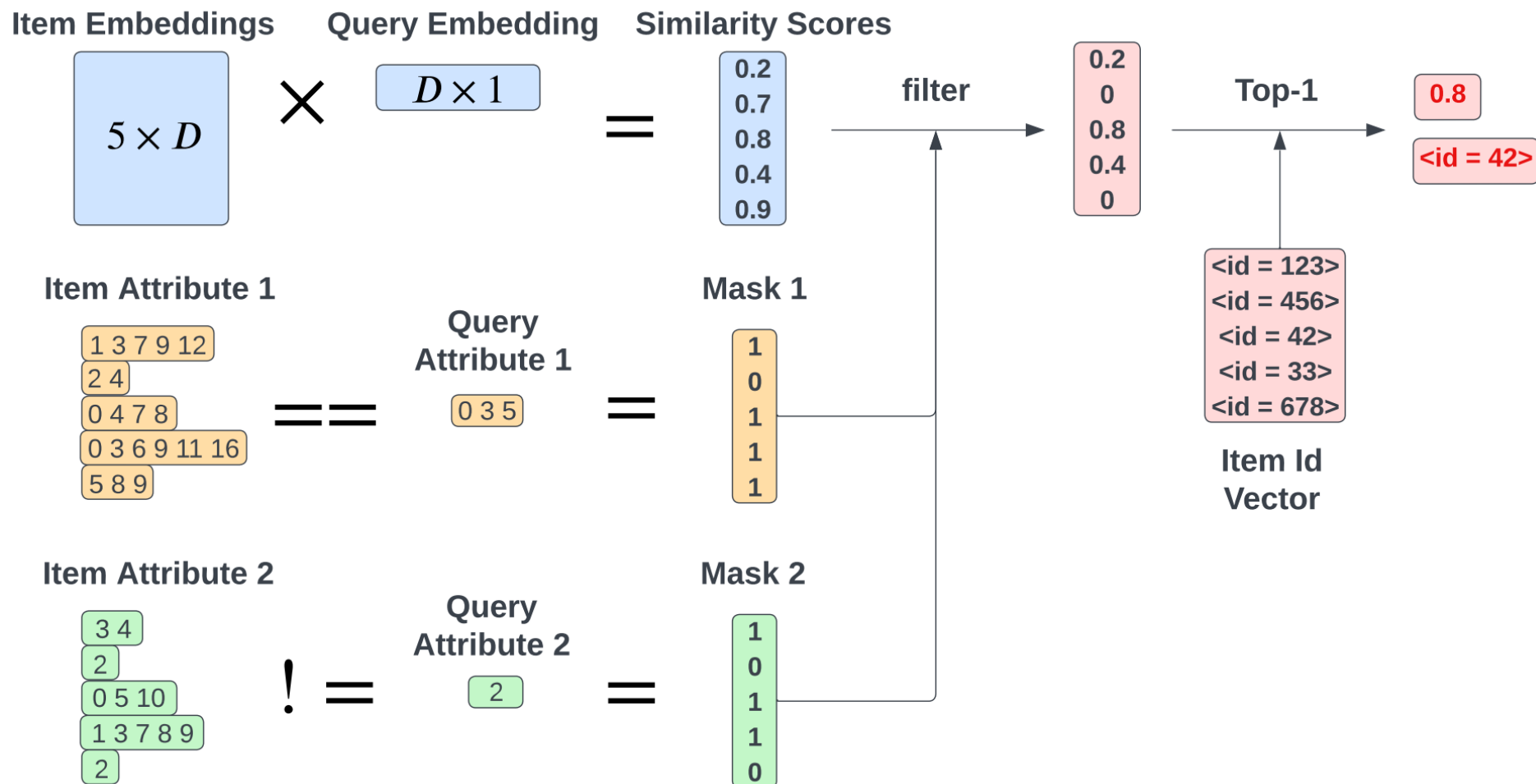
Поместили эмбединги документов и модель, вычисляющую эмбединг запроса, вместе на GPU.

Когда пришёл запрос:

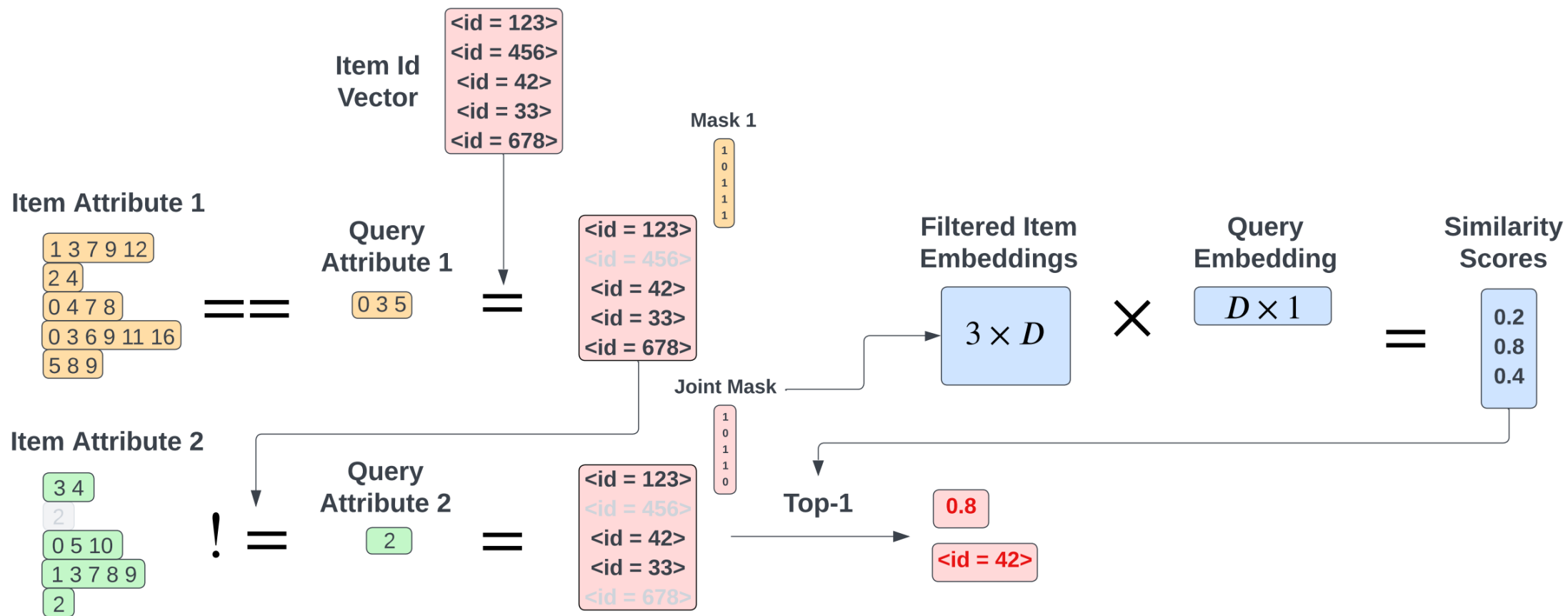
- Посчитали эмбединг запроса
- Пофильтровали все неподходящие документы исходя из настроенных пользователем фильтраций
- Посчитали для оставшихся документов все близости и выдали точный top K

С некоторыми оптимизациями на GPU можно уместить довольно большую базу документов.

GPU retrieval: постфилтрация



GPU retrieval: префильтрация



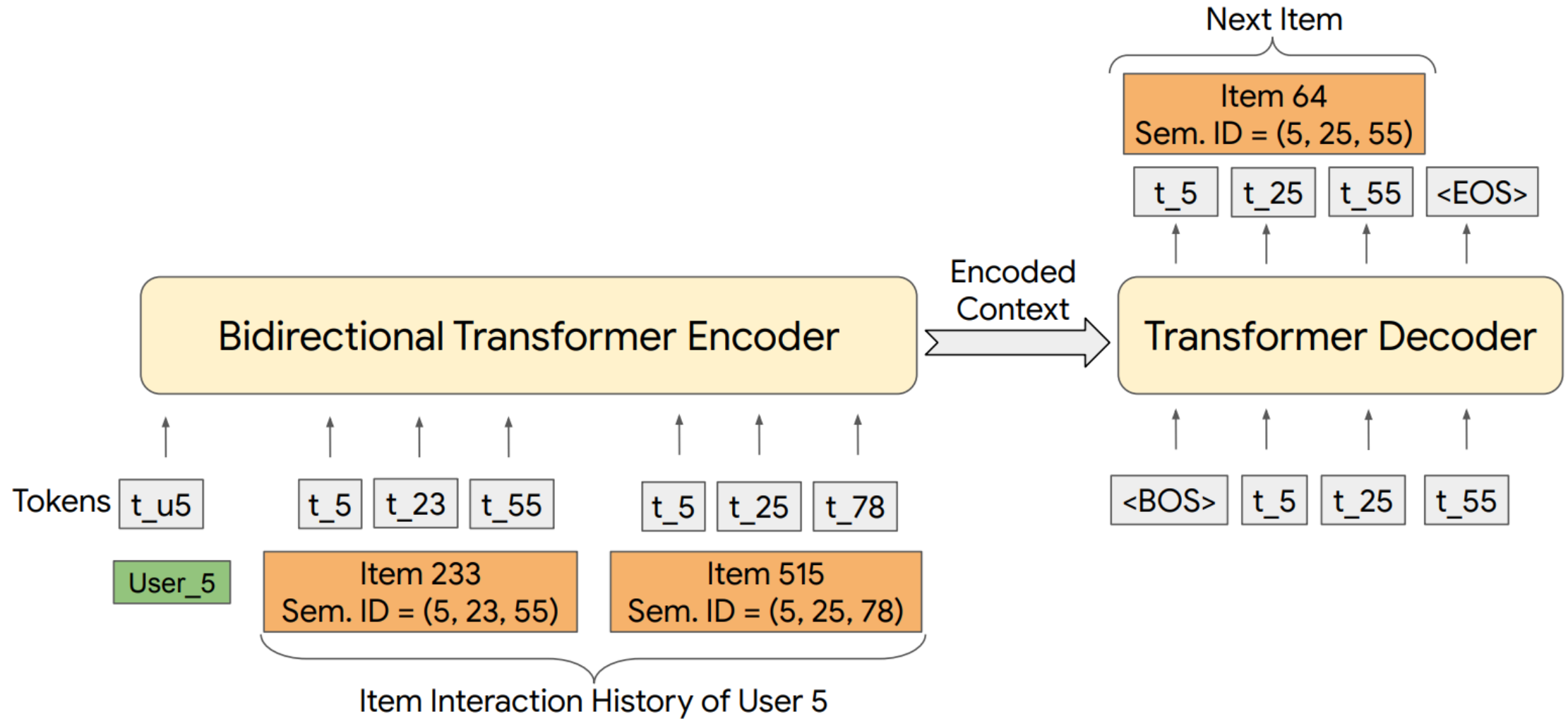
Generative retrieval

- Хотим уйти от двухбашенных моделей и парадигмы **encode + retrieve**
- Почему нам изначально были нужны индексы: из-за слишком большой кардинальности множества айтемов

Давайте сопоставим каждому документу последовательность айдишников низкой кардинальности:

- И научим модель генерировать эту последовательность в ответ на запрос

TIGER



Lecture summary

1. Обучаемые эмбединги – мощный, но хрупкий инструмент
2. Контентное кодирование – наш полезный inductive bias
3. Unsupervised representation learning – векторные представления можно получать разными способами
4. Кодирование пользователей – отдельная большая задача
5. Постепенно появляются альтернативы двухбашенным моделям, e.g. mixture of logits и generative retrieval

На семинаре

Обсудим различные аспекты обучения нейросетевых рекомендательных моделей.